TinyMAC: A 2×2 8-bit MAC Array

Huancheng Yang Jiangyue Chu

NYU Tandon School of Engineering

CogniChip Hackathon — Feb 20, 2026

Repository

GitHub: https://github.com/Torzirael36/Cognichip-Hackson.git

Outline

1. Introduction
2. Architecture and RTL
3. Using the Cognichip Platform
4. Verification and Simulation Evidence
5. Conclusion

Problem & Motivation

- Multiply-Accumulate (MAC) is the core operation behind matrix multiplication and convolution:

$$\text{acc} \leftarrow \text{acc} + (a \times b)$$

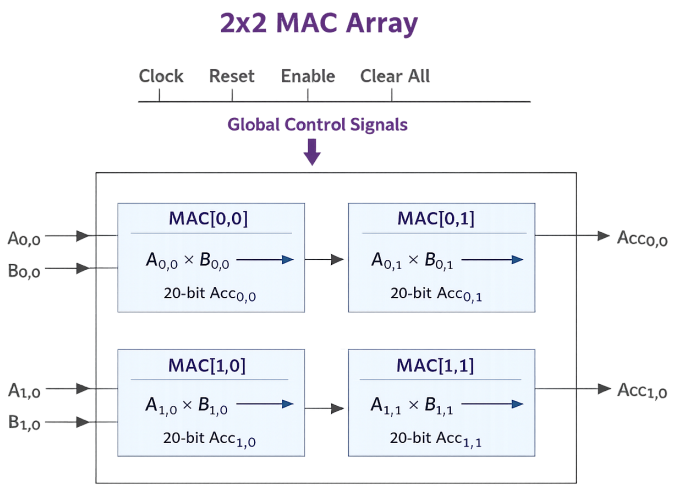
- Modern AI and DSP pipelines perform massive numbers of MAC operations; small compute blocks are the building units.
- **Goal:** build a compact, synthesizable **2×2 8-bit MAC array** with simple control and strong simulation evidence.

What we submit

RTL (SystemVerilog) + simulation testbench evidence (waveforms/logs) + final slides + demo video.

What We Built: TinyMAC 2×2 Overview

- A hierarchical, modular design with two layers:
  - **mac_unit**: single MAC with 8-bit inputs and a wide accumulator
  - **mac_array_2x2**: 4 MACs in parallel with global control signals
- Parallel compute: **4 MAC operations per cycle** (when enabled)
- Simple control interface: `enable` (pause/resume) and `clear_all` (reset accumulators)



MAC Semantics and Timing

- Operation:

$$\text{accumulator} \leftarrow \text{accumulator} + (a \times b)$$

- **Synchronous accumulation**: updates occur on the rising edge of `clock`.
- **Reset/Clear behavior**:
  - `reset` or `clear` forces accumulator to zero
  - `enable` gates accumulation (no update when disabled)
- Product width: 8-bit $\times$ 8-bit $\Rightarrow$ 16-bit product.

Design choice

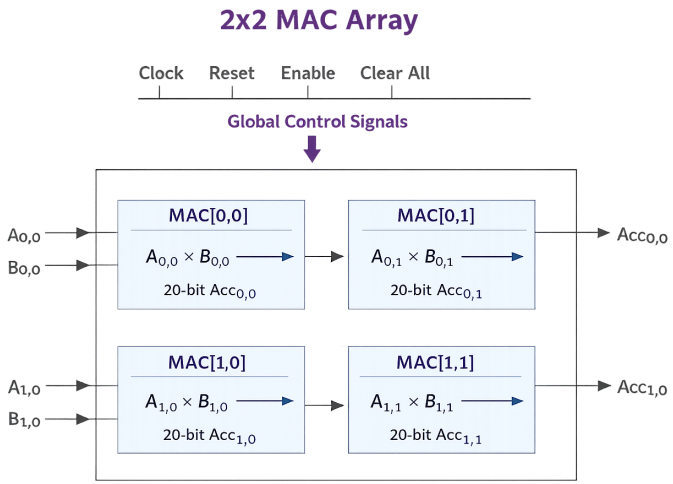
We use a wider accumulator to reduce overflow risk during repeated accumulation.

Accumulator Width Choice ($8\times8\rightarrow20$)

- Default parameters:
  - `DATA_WIDTH = 8`
  - `ACC_WIDTH = 20`
- Max single product: $255 \times 255 = 65025$ (fits in 16 bits).
- 20-bit accumulator supports substantially more headroom for repeated accumulation.
- This is a practical choice for small-kernel accumulation and repeated dot-product style workloads.

Module 2: `mac_array_2x2`

- A 2×2 grid of 4 independent MAC units:
  - MAC[0,0], MAC[0,1], MAC[1,0], MAC[1,1]
- Shared global signals:
  - `clock`, `reset`, `enable`, `clear_all`
- Independent channels:
  - each MAC has its own `a_ij`, `b_ij` inputs and `acc_ij` output



Key Features

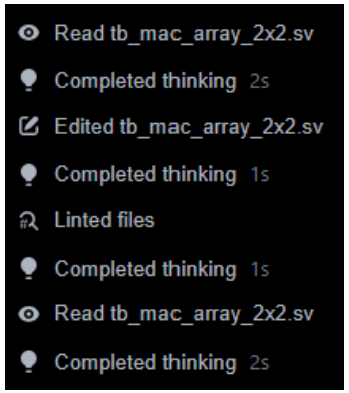
- **Parallel compute**: 4 MAC units operate simultaneously (4 MACs/cycle).
- **Global enable**: pause/resume accumulation without losing state.
- **Clear-all**: reset all accumulators for a new computation phase.
- **Modular and extensible**: parameterized units can scale to 4×4 or larger arrays.

How We Used Cognichip

- Generated and iterated on SystemVerilog RTL modules using the Cognichip workflow.
- Created/updated simulation targets (RTL + testbench + run configuration).
- Ran simulation to produce logs and waveform traces for correctness evidence.

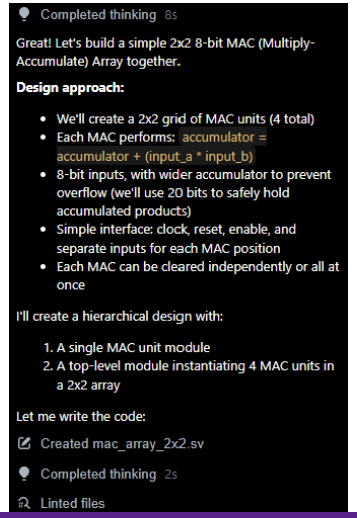
How We Used Cognichip for RTL Development

- We used Cognichip as an AI-assisted RTL co-design platform.
- Started with a high-level architectural description:
  - 2x2 MAC array
  - 8-bit inputs
  - 20-bit accumulator
  - Global control signals
- Cognichip generated synthesizable SystemVerilog automatically.
- Built hierarchical modules:
  - mac_unit
  - mac_array_2x2



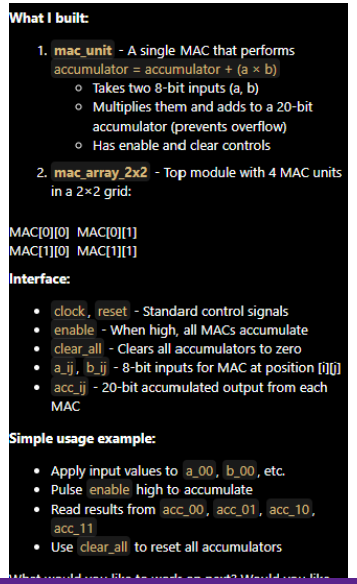
Prompt Engineering: Guiding the Design

- Instead of vague requests, we provided:
  - Explicit data widths
  - Accumulator overflow constraints
  - Clear/reset semantics
  - Hierarchical module structure
- Structured design approach:
  - Step 1: Define mac_unit
  - Step 2: Instantiate 2x2 grid
  - Step 3: Add control signals
- We treated Cognichip like a hardware co-engineer.



Iterative Refinement and Self-Verification

- After initial generation:
  - Ran lint checks
  - Requested improvements
  - Verified coding style
- Cognichip automatically:
  - Corrected structural issues
  - Ensured synthesizable constructs
  - Maintained clean hierarchical separation
- Achieved zero lint warnings.



What I built:

1. `mac_unit` - A single MAC that performs `accumulator = accumulator + (a × b)`
   - Takes two 8-bit inputs (a, b)
   - Multiplies them and adds to a 20-bit accumulator (prevents overflow)
   - Has enable and clear controls
2. `mac_array_2x2` - Top module with 4 MAC units in a 2×2 grid:

MAC[0][0] MAC[0][1]
MAC[1][0] MAC[1][1]

Interface:

- `clock`, `reset` - Standard control signals
- `enable` - When high, all MACs accumulate
- `clear_all` - Clears all accumulators to zero
- `a_ij`, `b_ij` - 8-bit inputs for MAC at position [i][j]
- `acc_ij` - 20-bit accumulated output from each MAC

Simple usage example:

- Apply input values to `a_00`, `b_00`, etc.
- Pulse `enable` high to accumulate
- Read results from `acc_00`, `acc_01`, `acc_10`, `acc_11`
- Use `clear_all` to reset all accumulators

Why Cognichip Is Powerful

- Rapid RTL generation:
  - Full 2x2 MAC array built in seconds
- Clean, parameterized, synthesizable code
- Supports hierarchical hardware abstraction
- Enables faster design exploration

Engineering Impact

Cognichip significantly reduces development time while maintaining hardware-level correctness and modularity.

From idea to working RTL in minutes.

Verification Strategy

- Testbench validates:
  - reset behavior
  - single accumulation
  - multi-cycle accumulation
  - parallel operation across 4 MACs
  - `enable` gating (pause/resume)
  - `clear_all` behavior
  - edge/extreme values (e.g., 0 and 255)
- We executed **9 test cases** covering both functional paths and corner cases.

TODO: If you have Python golden or randomized tests

Add: golden model checks, random vectors count (e.g., 100+), and pass rate summary.

Simulation Results: Waveform Evidence

Test Case Walkthrough: Multiply-Accumulate Behavior

- **Step 1: $3 \times 4 = 12$**
  - Inputs: `a_00 = 3`, `b_00 = 4`, `enable = 1`
  - Product: $3 \times 4 = 12 = 0x000C$
  - Accumulator update:

$$acc_{00} : 0x0000 \rightarrow 0x000C$$

  - `acc_00` matches `expected_00`
- **Step 2: $5 \times 6 = 30$**
  - $5 \times 6 = 30 = 0x001E$
  - Accumulation:

$$0x000C + 0x001E = 0x002A \ (42)$$

  - Waveform shows step-wise increase to `0x002A`
- **Step 3: $2 \times 3 = 6$**
  - $2 \times 3 = 6$
  - Accumulation:

$$0x002A + 0x0006 = 0x0030 \ (48)$$

Simulation Results: Log Evidence

- The cognichip runs the testbenches.
- All directed tests passed (9/9).
- The reset,enable signals are all tested.
- Logs record key checks and final PASS/FAIL summary.

Challenges, Lessons, and Future Work

- **Challenges:**
  - choosing accumulator width vs. overflow risk
  - defining clear/reset/enable semantics for robust TB checks
  - building strong simulation evidence (waveforms + logs)
- **Future work:**
  - scale to 4×4 / 8×8 arrays
  - add input buffering / register-mapped controller
  - optional FPGA demo and deeper pipelining

Repository

GitHub: https://github.com/Torzirael36/Cognichip-Hackson.git